

How Does PolyTrack Know a Run Was Driven by a Bot?

A deterministic-replay and statistical-detection analysis

Companion code: `bot_detection.py` (pure NumPy)

June 22, 2026

Abstract

PolyTrack (Kodub) stores every leaderboard time as a *recorded input sequence* replayed through a *bit-exact, fixed-timestep physics engine*. This single architectural choice does two very different jobs that are routinely confused. First, it makes times *verifiable*: you do not upload a number, you upload the keystrokes that produce it, and anyone can re-simulate them — so *forged times are impossible*. Second, it does *not*, by itself, distinguish a fast human from a fast bot: a bot that submits a legal, re-simulatable trace produces a fully valid run. Telling that the *driver* was an optimizer is a *statistical* problem, not a verification problem. Part I derives why determinism makes replay verification exact and delimits what it can and cannot catch. Part II builds a principled detector from the fingerprint of a bot’s input timing: hypothesis tests, a likelihood-ratio classifier, ROC analysis, and an information-theoretic bound on evasion, all demonstrated by the companion simulation.

Remark (Epistemic status). *The PolyTrack client architecture in Part I (deterministic 1 ms physics, an input-recording leaderboard at `vps.kodub.com`, ghost replays) is established fact, observed directly while building this repository’s headless simulator. The server-side detection policy in Part II is not published by Kodub; Part II is the detector the author would build, derived from first principles and validated in simulation. It is labeled as a model throughout.*

Part I — Verification: why a fake *time* is impossible

1 The run is the input, not the number

A PolyTrack run is integrated at a **fixed timestep** $\Delta t = 1$ ms (1000 Hz). The engine is a Bullet build compiled to Emscripten/WASM and is **deterministic**: given the same initial state and input stream it returns bit-identical results on every machine (this repository’s headless port reproduces the game’s own self-test exactly). Formally the simulation is a deterministic map

$$s_{k+1} = F(s_k, u_k), \quad s_0 = s_{\text{init}}, \quad (1)$$

where $s_k \in \mathbb{R}^n$ is the full physics state at frame k (the 227-byte car struct plus the rigid-body world) and $u_k \in \mathcal{U}$ is the control input on frame k — a bitmask over {left, right, throttle, brake}. The finish time is the first frame the goal predicate holds,

$$T = \Delta t \cdot \min\{k : \text{goal}(s_k)\}. \quad (2)$$

Because F is a pure function, T is a **deterministic functional of the input sequence alone**,

$$T = \mathcal{T}(u_{0:K}), \quad u_{0:K} = (u_0, u_1, \dots, u_K). \quad (3)$$

The leaderboard therefore need not trust a reported time: it stores $u_{0:K}$ (exactly what the ghost/replay feature plays back) and recomputes $\hat{T} = \mathcal{T}(u_{0:K})$ independently. **Validation is the assertion $\hat{T} = T_{\text{claimed}}$.**

2 What this catches, with certainty

Proposition 1 (Forged times and impossible physics are rejected w.p. 1). *Let an attacker submit a claimed time T^* with attached input $u_{0:K}$.*

- (a) *If $\mathcal{T}(u_{0:K}) \neq T^*$, the run is rejected. No input stream finishes in 0.0 s, so a forged time cannot be backed by a replay.*
- (b) *Any state edit (teleport, speed hack) producing s_{k+1}^{hack} for which no u satisfies $F(s_k, u) = s_{k+1}^{\text{hack}}$ is unreachable; the replay diverges from the claim and is rejected.*

Determinism is what turns this into a *proof* rather than a heuristic. Contrast a non-deterministic engine (variable timestep, RNG, float-order drift): there $\hat{T} \neq T$ even for honest runs, the equality test must be relaxed to a tolerance $|\hat{T} - T| < \varepsilon$, and that slack is precisely the room a cheater needs. The fixed 1 ms step removes the slack.

3 What this *cannot* catch — the crux

Replay verification checks that *the input produces the time*; it says nothing about *what produced the input*. A bot of the kind built in this repository does the honest thing: run the real physics F headless, search or learn an input sequence $u_{0:K}^*$ minimizing $\mathcal{T}$, then replay u^* in the browser. That submission satisfies $\hat{T} = T_{\text{claimed}}$ *exactly*, because it *was* generated by F . Verification thus reduces the problem to: *is this legal, re-simulatable input stream the kind of thing a human hand produces?* That is Part II.

Part II — Detection: the statistical fingerprint of a bot

4 Threat model and the core asymmetry

We observe one or more legal traces from a player and decide between

$$H_0 : \text{human driver} \quad \text{vs.} \quad H_1 : \text{automated driver.} \quad (4)$$

The leverage is a structural asymmetry in how the two generate $u_{0:K}$ (Table 1). Each row is measurable; none is individually conclusive; pooled, they separate cleanly (§7).

Table 1: Generative asymmetry between human and bot input.

	Human	Bot (search / RL)
Time base	continuous wall-clock	the 1 ms grid is the planning unit
Timing law	ex-Gaussian intervals	tight; can be frame-adjacent
Repeatability	high lap-to-lap variance	near-deterministic replay
Optimality	bounded by perception/dexterity	bounded only by search budget

5 Detector 1 — sub-frame phase (the sharpest single tell)

A human key event occurs at continuous wall-clock time τ . The input layer records frame $k = \lfloor \tau / \Delta t \rfloor$, but the residual **sub-frame phase**

$$\phi = \frac{\tau}{\Delta t} - \left\lfloor \frac{\tau}{\Delta t} \right\rfloor \pmod{1} \quad (5)$$

is, over many events, $\phi \sim \text{Uniform}[0, 1]$: muscle twitches do not align to the engine’s millisecond grid. A bot planning *on* the grid emits integer frames, so $\phi \equiv 0$. Test $H_0 : \phi \sim U[0, 1]$ with a binned chi-square over m bins,

$$X^2 = \sum_{j=1}^m \frac{(O_j - E_j)^2}{E_j}, \quad E_j = \frac{N}{m}, \quad X^2 \stackrel{H_0}{\sim} \chi_{m-1}^2. \quad (6)$$

The simulation (one run each) is decisive:

```
[D1 sub-frame phase chi^2] human stat= 12.33 p=0.195 | bot stat=1080.00 p=9.58e-227
```

The human is consistent with uniform ($p \approx 0.2$); the raw bot is rejected at $p \approx 10^{-227}$. *Caveat:* this tell evaporates if the bot dithers its phase before submitting (§8) — motivating multiple detectors.

6 Detector 2 — the inter-event interval law

Human inter-response intervals follow the **ex-Gaussian** law (a Gaussian motor component convolved with an exponential decision tail),

$$f_{\text{exG}}(x; \mu, \sigma, \tau) = \frac{1}{\tau} \exp\left(\frac{\mu - x}{\tau} + \frac{\sigma^2}{2\tau^2}\right) \Phi\left(\frac{x - \mu}{\sigma} - \frac{\sigma}{\tau}\right), \quad (7)$$

with a hard floor near 10–12 ms (one cannot voluntarily issue distinct presses faster). A bot has no such floor: an optimal plan routinely places control changes 1–3 ms apart. Compare empirical interval CDFs with a two-sample **Kolmogorov–Smirnov** statistic

$$D = \sup_x |F_{\text{obs}}(x) - F_{\text{human}}(x)|, \quad (8)$$

calibrated against a human reference:

```
[D2 interval-law KS] D=0.513 p=1.75e-14
```

A separation $D \approx 0.5$ is enormous. The most human-impossible scalar is the **fraction of frame-adjacent intervals** (≤ 3 ms); we use it directly in the classifier because, unlike phase, it *survives phase-dithering*: dithering moves events within a frame, it cannot widen a true 2 ms gap into a human 30 ms gap without changing the racing line.

7 Detector 3, Detector 4, and pooling

Across R repeated attempts, lap time has **coefficient of variation**

$$CV = \frac{\text{std}(T_1, \dots, T_R)}{\text{mean}(T_1, \dots, T_R)}, \quad (9)$$

with humans near 5% and a replayed bot at $\lesssim 0.3\%$. Detector 4 is the action **entropy** $H = -\sum_a p_a \log_2 p_a$: a clean optimal plan is slightly lower entropy than fidgety human input.

Pooling. The Neyman–Pearson optimal test combines features by their log-likelihood ratio,

$$\Lambda(x) = \log \frac{p(x | H_1)}{p(x | H_0)} \underset{H_0}{\overset{H_1}{\gtrless}} \eta, \quad (10)$$

and a logistic-regression score is the LLR under a Gaussian-feature linear model — what the companion code fits over $x = (X_{\text{phase}}^2, \text{frac frame-adjacent}, H, CV)$. On held-out players:

```
POOLED LLR CLASSIFIER (mimic=off)
  chi2_phase           +2.831
  frac_frame_adjacent  +2.533
  action_entropy       -1.105
  lap_CV               -0.878
held-out ROC AUC = 1.0000  accuracy@0.5 = 1.000
```

8 Evasion and the detectability limit

Run the experiment with `--mimic`, where the bot adds sub-frame jitter and inflates CV to look human:

```
POOLED LLR CLASSIFIER (mimic=ON)
  chi2_phase           -0.083    <- D1 NEUTRALIZED
  frac_frame_adjacent  +4.619    <- now carries the decision
  action_entropy       -2.101
  lap_CV               -0.267
held-out ROC AUC = 1.0000
```

Phase dithering kills D1 (weight $\rightarrow 0$, as theory predicts), but the classifier re-weights onto frame-adjacent intervals and AUC stays at 1.0. The reason is structural: mimicking human timing requires emitting intervals *wider* than the optimal plan, which changes the trajectory and costs time — a Pareto wall between *looking human* and *being world-record fast*.

Theorem 1 (Evasion vs. optimality, informally). *Let P_0, P_1 be the human and (disguised) bot distributions over observable input features. By the Chernoff–Stein lemma, the number of runs n needed to separate H_0, H_1 at type-II error α (fixed type-I) satisfies*

$$n \gtrsim \frac{1}{\text{KL}(P_1 \| P_0)} \log \frac{1}{\alpha}. \quad (11)$$

Evasion seeks $\text{KL}(P_1 \| P_0) \rightarrow 0$. A world-record bot cannot: WR times force low-CV, frame-tight inputs that keep KL bounded away from 0, so n stays small (a few runs suffice). A bot willing to discard the record can shrink KL arbitrarily and become undetectable from timing alone.

Corollary (the project’s tension). The more superhuman the time, the easier the bot is to catch.

9 Other server-side signals (beyond the trace)

A real anti-cheat also has cheap side channels that raise KL in directions a bot cannot flatten: (i) *holistic line plausibility* — trajectories optimal to sub-frame tolerance on *every* corner; (ii) *client integrity* — headless clients, missing render-loop/mouse-move entropy, replays without a live session; (iii) *rate & novelty* — a flood of WR-grade times via a sequential probability ratio test (SPRT) over the submission stream; (iv) *cross-player collision* — byte-identical or trivially perturbed traces across accounts.

10 The decision pipeline

```
submission (claimed time + input trace u.0:K)
↓
PART I -- VERIFICATION (exact, P=1): re-simulate  $\hat{T} = T(u.0:K)$  with F;
  reject if  $\hat{T} \neq T_{\text{claimed}}$  or state illegal
  ↓ legal, re-simulatable run
PART II -- DETECTION (statistical):
  D1 sub-frame phase  $\chi^2$  vs U[0,1)
  D2 interval law: KS / frac frame-adjacent
  D3 cross-run CV    D4 action entropy
  pool  $\rightarrow$  LLR  $\Lambda(x) \geq \eta \rightarrow$  human / bot
  + side channels, SPRT over history
```

Verification is a proof; detection is a likelihood ratio. Conflating them is the most common mistake in “how does the game catch bots” discussions.

11 Reproducing the results

```
cd polytrack-ai/docs
python bot_detection.py          # tables in Sec. 5-7
python bot_detection.py --mimic  # the evasion result, Sec. 9
python bot_detection.py --plot   # feature histograms (needs matplotlib)
```

Only NumPy is required; the chi-square, KS, and incomplete-gamma routines are implemented from scratch in the companion file (no SciPy dependency).

12 Limitations

Part II is a *model* of a detector, validated on *synthetic* generators, not on PolyTrack’s unpublished server logic or on captured traces. If real human input is grid-aligned (e.g. a 60Hz input poll quantizing events), D1’s power drops and weight shifts to D2 — precisely the robustness the `--mimic` experiment shows. The AUC of 1.0 reflects well-separated synthetic classes; read it as “these features are jointly informative,” not a real-world catch rate. The honest takeaway holds regardless: a verifiable run is easy; a verifiable run that is statistically indistinguishable from a human *and* still a world record is provably hard.